

TP : Détection de Piétons par Approche Hybride

Deep Learning (YOLO) + Vision Classique (Sobel, Hough)

Maryam Boufous

3 mars 2026

Niveau : Informatique / Vision par Ordinateur

Prérequis : C++, notions de traitement d'images, algèbre linéaire de base

Objectifs Pédagogiques

- Comprendre les principes mathématiques de la détection de contours (Sobel)
- Maîtriser la transformée de Hough pour la détection de lignes
- Appréhender l'architecture et le fonctionnement de YOLO
- Implémenter un système complet de détection de piétons
- Analyser les performances d'une approche hybride

Table des matières

1	Introduction Théorique	4
1.1	La Détection de Piétons : Un Défi Majeur	4
1.2	Approche Hybride Proposée	4
2	Fondements Mathématiques	4
2.1	Le Gradient d'Image et l'Opérateur de Sobel	4
2.1.1	Définition du Gradient	4
2.1.2	Discrétisation avec Sobel	5
2.1.3	Interprétation des Noyaux	5
2.1.4	Propriétés de l'Opérateur de Sobel	5
2.2	La Transformée de Hough pour les Lignes	5
2.2.1	Principe Fondamental	5
2.2.2	Espace de Hough	6
2.2.3	Algorithme de la Transformée de Hough	6
2.2.4	Transformée de Hough Probabiliste	6
2.3	YOLO (You Only Look Once)	6
2.3.1	Architecture du Réseau	6
2.3.2	Fonction de Prédiction	7
2.3.3	Fonction de Coût	7
2.4	Suppression Non-Maximale (NMS)	7
2.4.1	Intersection over Union (IoU)	7
2.4.2	Algorithme NMS	7
3	Architecture du Système	8
3.1	Vue d'Ensemble	8
3.2	Flux de Données	8
4	Implémentation Détaillée	8
4.1	En-têtes et Déclarations	8
4.2	Fonction de Détection de Contours Sobel	9
4.3	Fonction d'Analyse de Structure Verticale	10
4.4	Fonction de Détection YOLO	12
4.5	Fonction Principale	14
5	Compilation et Exécution	18
5.1	Compilation avec CMake	18
5.2	Compilation Directe	18
5.3	Exécution	18
6	Exercices et Analyses	19
6.1	Exercice 1 : Étude des Paramètres de Sobel	19
6.2	Exercice 2 : Optimisation des Paramètres Hough	19
6.3	Exercice 3 : Métriques de Performance	19
6.4	Exercice 4 : Détection de Têtes par HoughCircles	19

7	Annexes Techniques	20
7.1	Annexe A : Classes COCO	20
7.2	Annexe B : Codes d'Erreur OpenCV	20
7.3	Annexe C : Optimisations Possibles	21
8	Conclusion	21

1 Introduction Théorique

1.1 La Détection de Piétons : Un Défi Majeur

La détection de piétons est un problème fondamental en vision par ordinateur qui consiste à localiser précisément les personnes dans une image. Ce problème présente plusieurs défis :

- **Variabilité d'apparence** : Vêtements, postures, orientations différentes
- **Conditions d'éclairage** : Ombres, contre-jour, faible luminosité
- **Encombrement** : Piétons partiellement cachés, groupes
- **Échelle** : Piétons proches (grands) ou lointains (petits)

1.2 Approche Hybride Proposée

Notre système combine deux paradigmes complémentaires :

1. **Deep Learning (YOLOv4-tiny)** : Détection rapide et robuste basée sur l'apprentissage
2. **Vision Classique (Sobel + Hough)** : Vérification géométrique basée sur des modèles explicites

Cette approche permet de bénéficier de la puissance du deep learning tout en conservant l'interprétabilité des méthodes classiques.

2 Fondements Mathématiques

2.1 Le Gradient d'Image et l'Opérateur de Sobel

2.1.1 Définition du Gradient

Pour une image $I(x, y)$ considérée comme une fonction continue, le gradient est défini par :

$$\nabla I(x, y) = \begin{pmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{pmatrix}$$

La magnitude du gradient indique la force du contour :

$$|\nabla I| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

L'orientation du gradient donne la direction du contour :

$$\theta = \arctan\left(\frac{\partial I}{\partial y} / \frac{\partial I}{\partial x}\right)$$

2.1.2 Discrétisation avec Sobel

En traitement d'images numérique, on approxime les dérivées par des différences finies pondérées. L'opérateur de Sobel utilise deux noyaux de convolution 3x3 :

Noyau horizontal (détection des contours verticaux) :

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Noyau vertical (détection des contours horizontaux) :

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

2.1.3 Interprétation des Noyaux

Le noyau G_x donne plus de poids aux pixels centraux (coefficient 2) et effectue une différence entre les colonnes de gauche et de droite. Cela correspond à une approximation de la dérivée horizontale avec un lissage vertical.

Calcul du gradient horizontal en un point (i, j) :

$$\begin{aligned} G_x(i, j) = & (-1) \cdot I(i-1, j-1) + 0 \cdot I(i-1, j) + (+1) \cdot I(i-1, j+1) \\ & + (-2) \cdot I(i, j-1) + 0 \cdot I(i, j) + (+2) \cdot I(i, j+1) \\ & + (-1) \cdot I(i+1, j-1) + 0 \cdot I(i+1, j) + (+1) \cdot I(i+1, j+1) \end{aligned}$$

2.1.4 Propriétés de l'Opérateur de Sobel

- **Séparabilité** : Le noyau peut être décomposé en un produit de convolution d'un lissage et d'une dérivée
- **Robustesse au bruit** : La pondération centrale réduit l'impact du bruit
- **Isotropie approximative** : La réponse est similaire dans toutes les directions

2.2 La Transformée de Hough pour les Lignes

2.2.1 Principe Fondamental

La transformée de Hough repose sur le principe de dualité point-droite. Une ligne dans l'espace image peut être représentée par deux paramètres.

Représentation paramétrique (normale) :

$$\rho = x \cos \theta + y \sin \theta$$

où :

- ρ est la distance de la ligne à l'origine
- θ est l'angle du vecteur normal à la ligne

2.2.2 Espace de Hough

Chaque point (x, y) de l'image correspond à une sinusoïde dans l'espace (ρ, θ) :

$$\rho(\theta) = x \cos \theta + y \sin \theta$$

Les points alignés dans l'image produisent des sinusoïdes qui se coupent en un point commun dans l'espace de Hough.

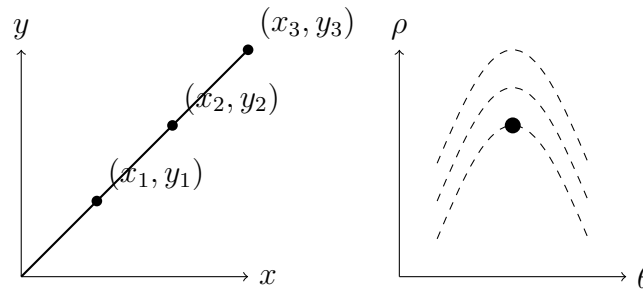


FIGURE 1 – Dualité entre l'espace image (gauche) et l'espace de Hough (droite)

2.2.3 Algorithme de la Transformée de Hough

1. Discrétiser l'espace (ρ, θ) en accumulateur $A(\rho, \theta)$
2. Pour chaque point de contour (x, y) :
 - Pour chaque angle θ de 0 à π :
 - Calculer $\rho = x \cos \theta + y \sin \theta$
 - Incrémenter $A(\rho, \theta)$
3. Les maxima locaux de A correspondent aux lignes détectées

2.2.4 Transformée de Hough Probabiliste

La version utilisée dans notre code (HoughLinesP) est optimisée :

- Ne prend qu'un sous-ensemble aléatoire de points
- Détecte des segments de ligne (avec points de début et fin)
- Plus rapide et plus précise pour les applications réelles

2.3 YOLO (You Only Look Once)

2.3.1 Architecture du Réseau

YOLOv4-tiny est un réseau de neurones convolutif (CNN) composé de :

- **Backbone** : CSPDarknet53-tiny (extraction de caractéristiques)
- **Neck** : FPN (Feature Pyramid Network) pour la fusion multi-échelle
- **Head** : Couches de prédiction pour les boîtes englobantes

2.3.2 Fonction de Prédiction

Pour chaque cellule de la grille, YOLO prédit :

1. **Coordonnées de la boîte** : t_x, t_y, t_w, t_h
2. **Score d'objectivité** : $P(\text{objet}) \times \text{IoU}$
3. **Probabilités de classe** : $P(\text{classe}_i | \text{objet})$

Conversion en coordonnées image :

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

$$b_w = p_w e^{t_w}$$

$$b_h = p_h e^{t_h}$$

où (c_x, c_y) est le coin de la cellule et (p_w, p_h) les dimensions de l'ancree.

2.3.3 Fonction de Coût

L'entraînement minimise une fonction de coût multi-parties :

$$\mathcal{L} = \mathcal{L}_{\text{coord}} + \mathcal{L}_{\text{conf}} + \mathcal{L}_{\text{class}}$$

2.4 Suppression Non-Maximale (NMS)

2.4.1 Intersection over Union (IoU)

L'IoU mesure le chevauchement entre deux boîtes :

$$\text{IoU} = \frac{\text{Aire}(B_1 \cap B_2)}{\text{Aire}(B_1 \cup B_2)}$$

2.4.2 Algorithme NMS

1. Trier toutes les détections par score de confiance
2. Initialiser la liste des détections finales $D = \emptyset$
3. Pour chaque détection d (du meilleur au moins bon) :
 - Si $\text{IoU}(d, d') < \text{seuil}$ pour tout $d' \in D$:
 - Ajouter d à D
 - Sinon : rejeter d (considéré comme doublon)

3 Architecture du Système

3.1 Vue d'Ensemble

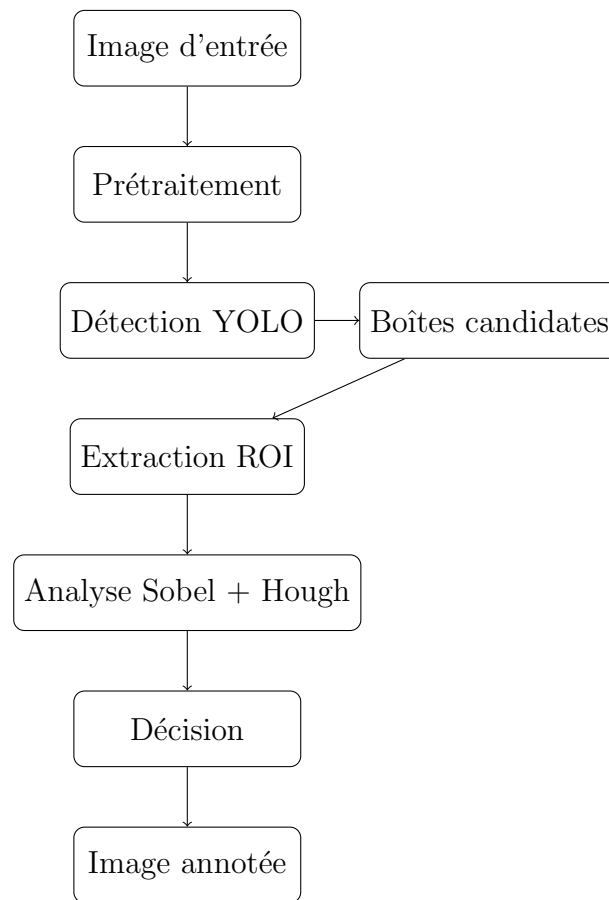


FIGURE 2 – Pipeline complète du système de détection

3.2 Flux de Données

1. **Entrée** : Image RGB (format BGR dans OpenCV)
2. **Prétraitement** : Redimensionnement, normalisation
3. **Détection YOLO** : Inférence sur le réseau
4. **Post-traitement** : NMS, filtrage par confiance
5. **Vérification** : Pour chaque boîte, analyse Sobel+Hough
6. **Visualisation** : Annotations avec code couleur

4 Implémentation Détaillée

4.1 En-têtes et Déclarations

```
1 #ifndef DETECTION_PIETONS_H
2 #define DETECTION_PIETONS_H
3
4 #include <opencv2/opencv.hpp>
```

```

5 #include <opencv2/dnn.hpp>
6 #include <iostream>
7 #include <vector>
8 #include <string>
9 #include <cmath>
10
11 using namespace cv;
12 using namespace cv::dnn;
13 using namespace std;
14
15 // Constantes de configuration
16 namespace Config {
17     const string MODEL_CFG      = "models/yolov4-tiny.cfg";
18     const string MODEL_WEIGHTS = "models/yolov4-tiny.weights";
19     const float  CONF_THRESHOLD = 0.35f;
20     const float  NMS_THRESHOLD  = 0.40f;
21     const int    INPUT_WIDTH   = 416;
22     const int    INPUT_HEIGHT  = 416;
23     const int    SOBEL_THRESHOLD = 50;
24     const double MIN_VERTICAL_RATIO = 0.30;
25 }
26
27 // Déclarations des fonctions
28 Mat getSobelEdges(const Mat& roi);
29 bool hasStrongVerticalStructure(const Mat& roi);
30 vector<Rect> detectPedestriansYOLO(const Mat& image, Net& net, vector<float>& confidences);
31 void drawDetections(Mat& image, const vector<Rect>& boxes,
32                        const vector<float>& confidences, const vector<bool>
33                        & verified);
34 #endif

```

Listing 1 – detection.hpp - Fichier d'en-tête

Explication détaillée :

- **Lignes 27-31** : Prototypes des fonctions principales
 - `getSobelEdges` : Calcule la carte de contours Sobel
 - `hasStrongVerticalStructure` : Analyse la verticalité
 - `detectPedestriansYOLO` : Exécute la détection YOLO
 - `drawDetections` : Affiche les résultats

4.2 Fonction de Détection de Contours Sobel

```

1 Mat getSobelEdges(const Mat& roi) {
2     // Vérification de la validité de la région d'intérêt
3     if (roi.empty()) {
4         return Mat();
5     }
6
7     // Variables intermédiaires
8     Mat gray, blurred, gradX, gradY;
9     Mat absGradX, absGradY, edges;
10
11     // Etape 1: Conversion en niveaux de gris
12     // La détection de contours est plus efficace sur une seule channel

```

```

13     cvtColor(roi, gray, COLOR_BGR2GRAY);
14
15     // Etape 2: Reduction du bruit par filtre gaussien
16     // Le bruit peut creer des faux contours, on lisse donc l'image
17     // Taille du noyau: 5x5, ecart-type calcule automatiquement
18     GaussianBlur(gray, blurred, Size(5, 5), 0);
19
20     // Etape 3: Calcul des gradients avec Sobel
21     // Parametres: source, destination, profondeur (16 bits signe),
22     // ordre de derivation en x (1), ordre en y (0), taille du noyau (3)
23     Sobel(blurred, gradX, CV_16S, 1, 0, 3);
24     Sobel(blurred, gradY, CV_16S, 0, 1, 3);
25
26     // Etape 4: Conversion en valeurs absolues 8-bit
27     // Les gradients peuvent etre negatifs, on prend la valeur absolue
28     convertScaleAbs(gradX, absGradX);
29     convertScaleAbs(gradY, absGradY);
30
31     // Etape 5: Combinaison ponderee des gradients
32     // On donne autant d'importance aux contours verticaux qu'
horizontaux
33     // Formule: edges = 0.5*|Gx| + 0.5*|Gy|
34     addWeighted(absGradX, 0.5, absGradY, 0.5, 0, edges);
35
36     // Etape 6: Seuillage pour obtenir une image binaire
37     // On garde seulement les contours significatifs (>50)
38     // THRESH_BINARY: valeur max si > seuil, 0 sinon
39     threshold(edges, edges, Config::SOBEL_THRESHOLD, 255, THRESH_BINARY)
40 ;
41     return edges;
42 }

```

Listing 2 – Implémentation de getSobelEdges()

Analyse mathématique de la fonction :

Pour un pixel (i, j) , la valeur après Sobel est :

$$E(i, j) = \frac{1}{2} \left| \sum_{m=-1}^1 \sum_{n=-1}^1 G_x(m, n) \cdot I(i + m, j + n) \right| + \frac{1}{2} \left| \sum_{m=-1}^1 \sum_{n=-1}^1 G_y(m, n) \cdot I(i + m, j + n) \right|$$

Après seuillage :

$$E_{\text{binaire}}(i, j) = \begin{cases} 255 & \text{si } E(i, j) \geq 50 \\ 0 & \text{sinon} \end{cases}$$

4.3 Fonction d'Analyse de Structure Verticale

```

1 bool hasStrongVerticalStructure(const Mat& roi, double minVerticalRatio
= Config::MIN_VERTICAL_RATIO) {
2     // Verification preliminaire
3     if (roi.empty() || roi.rows < 20 || roi.cols < 10) {
4         // Region trop petite pour une analyse fiable
5         return false;
6     }
7

```

```

8 // Etape 1: Obtention de la carte de contours par Sobel
9 Mat edges = getSobelEdges(roi);
10 if (edges.empty()) {
11     return false;
12 }
13
14 // Etape 2: Detection des lignes par transformee de Hough
15 // probabiliste
16 vector<Vec4i> lines;
17 HoughLinesP(
18     edges, // Image d'entree (binaire)
19     lines, // Vecteur de sortie pour les lignes
20     1, // Resolution en distance (pixels)
21     CV_PI/180, // Resolution angulaire (1 degre)
22     50, // Seuil de l'accumulateur
23     35, // Longueur minimale d'un segment
24     12 // Ecart maximal entre points d'un segment
25 );
26
27 if (lines.empty()) {
28     return false;
29 }
30
31 // Etape 3: Analyse de l'orientation des lignes
32 int verticalCount = 0;
33 for (const auto& l : lines) {
34     // Coordonnees des extremités du segment
35     Point p1(l[0], l[1]);
36     Point p2(l[2], l[3]);
37
38     // Calcul des composantes
39     int dx = abs(p2.x - p1.x); // Composante horizontale
40     int dy = abs(p2.y - p1.y); // Composante verticale
41
42     // Calcul de l'angle par rapport a l'horizontale
43     double angle = atan2(dy, dx) * 180 / CV_PI;
44
45     // Une ligne est consideree verticale si son angle > 65 degres
46     // (correspond a dy > dx * 2.14)
47     if (dy > dx * 2.5) { // Equivalent a angle > 68 degres
48         verticalCount++;
49
50         // Debug: afficher les caracteristiques de la ligne
51         // cout << "Ligne verticale: angle=" << angle << " ,
52         // longueur=" << dy << endl;
53     }
54 }
55
56 // Etape 4: Calcul du ratio de verticalite
57 double verticalRatio = static_cast<double>(verticalCount) / lines.
58 size();
59
60 // Etape 5: Decision
61 // On retourne vrai si le ratio depasse le seuil
62 return verticalRatio >= minVerticalRatio;
63 }

```

Listing 3 – Implémentation de hasStrongVerticalStructure()

Interprétation géométrique :

La condition de verticalité $dy > 2.5 \times dx$ correspond à un angle θ tel que :

$$\tan(\theta) = \frac{dy}{dx} > 2.5 \Rightarrow \theta > \arctan(2.5) \approx 68.2^\circ$$

Le ratio de verticalité est défini par :

$$R_v = \frac{N_{\text{verticales}}}{N_{\text{total}}}$$

Un piéton est considéré comme valide si $R_v \geq 0.30$, soit au moins 30% des lignes détectées sont verticales.

4.4 Fonction de Détection YOLO

```

1 vector<Rect> detectPedestriansYOLO(const Mat& image, Net& net, vector<
  float>& confidences) {
2     vector<Rect> boxes;
3     vector<float> scores;
4     vector<int> classIds;
5
6     // Etape 1: Creation du blob d'entree
7     // Le blob est un tenseur 4D: [batch, channels, height, width]
8     Mat blob = blobFromImage(
9         image,                                     // Image source
10        1/255.0,                                   // Facteur de normalisation (
mise a l'echelle [0,1])
11        Size(Config::INPUT_WIDTH, Config::INPUT_HEIGHT), // Taille cible
12        Scalar(0,0,0),                             // Valeur de soustraction (
aucune ici)
13        true,                                       // swapRB: echanger R et B (
OpenCV utilise BGR)
14        false                                     // crop: ne pas rogner l'image
15    );
16
17    // Etape 2: Passage dans le reseau
18    net.setInput(blob);
19    vector<Mat> outputs;
20    net.forward(outputs, net.getUnconnectedOutLayersNames());
21
22    // Etape 3: Parcours des sorties
23    for (size_t i = 0; i < outputs.size(); ++i) {
24        // Chaque sortie est une matrice de forme [nb_detections, 85]
25        // Les 85 colonnes: [cx, cy, w, h, objectness, scores_classe...]
26        float* data = (float*)outputs[i].data;
27
28        for (int row = 0; row < outputs[i].rows; ++row) {
29            // Extraction du score d'objectivite
30            float objectConfidence = data[4];
31
32            if (objectConfidence > Config::CONF_THRESHOLD) {
33                // Extraction des scores de classe
34                Mat scores = outputs[i].row(row).colRange(5, outputs[i].
cols);
35                Point classIdPoint;
36                double classConfidence;

```

```

37
38         // Recherche de la classe avec le score maximum
39         minMaxLoc(scores, nullptr, &classConfidence, nullptr, &
classIdPoint);
40
41         // Verification: classe 0 = personne dans COCO
42         if (classIdPoint.x == 0) {
43             // Conversion des coordonnees normalisees en pixels
44             float centerX = data[0] * image.cols;
45             float centerY = data[1] * image.rows;
46             float width   = data[2] * image.cols;
47             float height  = data[3] * image.rows;
48
49             // Conversion centre+taille -> coin superieur gauche
50             int left = max(0, (int)(centerX - width/2));
51             int top  = max(0, (int)(centerY - height/2));
52
53             // Ajustement pour rester dans l'image
54             width  = min(width, (float)(image.cols - left));
55             height = min(height, (float)(image.rows - top));
56
57             // Stockage de la detection
58             boxes.push_back(Rect(left, top, (int)width, (int)
height));
59             confidences.push_back(objectConfidence *
classConfidence);
60             classIds.push_back(classIdPoint.x);
61         }
62     }
63
64     // Passage a la detection suivante
65     data += outputs[i].cols;
66 }
67 }
68
69 // Etape 4: Application de la NMS
70 vector<int> indices;
71 NMSBoxes(boxes, confidences, Config::CONF_THRESHOLD, Config::
NMS_THRESHOLD, indices);
72
73 // Etape 5: Filtrage des detections par NMS
74 vector<Rect> finalBoxes;
75 vector<float> finalConfidences;
76
77 for (int idx : indices) {
78     finalBoxes.push_back(boxes[idx]);
79     finalConfidences.push_back(confidences[idx]);
80 }
81
82 // Mise a jour des confidences
83 confidences = finalConfidences;
84
85 return finalBoxes;
86 }

```

Listing 4 – Implémentation de detectPedestriansYOLO()

Format de sortie de YOLO :

Chaque détection est représentée par un vecteur de 85 valeurs :

$$\mathbf{d} = [t_x, t_y, t_w, t_h, P_o, P_1, P_2, \dots, P_{80}]$$

où :

- t_x, t_y : Coordonnées du centre relatives à la cellule
- t_w, t_h : Dimensions relatives aux ancres
- P_o : Probabilité qu'un objet soit présent
- P_i : Probabilité que l'objet soit de classe i

La confiance finale pour une classe c est :

$$\text{Conf}_c = P_o \times P_c$$

4.5 Fonction Principale

```

1 int main(int argc, char** argv) {
2     // =====
3     // PHASE 1: INITIALISATION ET VERIFICATION DES ARGUMENTS
4     // =====
5
6     // Verification du nombre d'arguments
7     if (argc < 2) {
8         cout << "\n
9
10         " << endl;
11         cout << "          DETECTEUR DE PIETONS - MODE D'EMPLOI          " <<
12         endl;
13         cout << "\n" << endl;
14         cout << "Usage: " << argv[0] << " <image_path> [output_path]\n"
15         << endl;
16         cout << "Exemples:" << endl;
17         cout << " " << argv[0] << " data/test.jpg" << endl;
18         cout << " " << argv[0] << " data/test.jpg results/out.png\n" <<
19         endl;
20         cout << "Fichiers requis:" << endl;
21         cout << " - models/yolov4-tiny.cfg" << endl;
22         cout << " - models/yolov4-tiny.weights\n" << endl;
23         return -1;
24     }
25
26     // Determination des chemins d'entree/sortie
27     string inputPath = argv[1];
28     string outputPath = (argc >= 3) ? argv[2] : "results/resultat.png";
29
30     // =====
31     // PHASE 2: CHARGEMENT DE L'IMAGE
32     // =====
33
34     cout << "\n[1/6] Chargement de l'image: " << inputPath << endl;
35     Mat image = imread(inputPath);
36
37     if (image.empty()) {

```

```
33     cerr << "ERREUR: Impossible de lire l'image: " << inputPath <<
endl;
34     cerr << "Verifiez que le fichier existe et est accessible." <<
endl;
35     return -1;
36 }
37
38 cout << "          - Dimensions: " << image.cols << "x" << image.rows <<
endl;
39 cout << "          - Format: " << image.type() << " (8UC3 = BGR)" <<
endl;
40
41 // =====
42 // PHASE 3: CHARGEMENT DU MODELE YOLO
43 // =====
44
45 cout << "\n[2/6] Chargement du modele YOLOv4-tiny..." << endl;
46
47 Net net = readNetFromDarknet(Config::MODEL_CFG, Config::
MODEL_WEIGHTS);
48
49 if (net.empty()) {
50     cerr << "ERREUR: Impossible de charger le modele YOLO" << endl;
51     cerr << "Verifiez la presence des fichiers:" << endl;
52     cerr << "          - " << Config::MODEL_CFG << endl;
53     cerr << "          - " << Config::MODEL_WEIGHTS << endl;
54     return -1;
55 }
56
57 // Configuration du backend (CPU par default)
58 net.setPreferableBackend(DNN_BACKEND_OPENCV);
59 net.setPreferableTarget(DNN_TARGET_CPU);
60
61 cout << "          - Backend: OpenCV" << endl;
62 cout << "          - Target: CPU" << endl;
63
64 // =====
65 // PHASE 4: DETECTION YOLO
66 // =====
67
68 cout << "\n[3/6] Detection des pietons par YOLO..." << endl;
69
70 vector<float> confidences;
71 vector<Rect> boxes = detectPedestriansYOLO(image, net, confidences);
72
73 cout << "          - " << boxes.size() << " detection(s) trouvee(s)" <<
endl;
74
75 if (boxes.empty()) {
76     cout << "\nAucun pieton detecte. Fin du programme." << endl;
77     return 0;
78 }
79
80 // =====
81 // PHASE 5: VERIFICATION PAR SOBEL + HOUGH
82 // =====
83
84 cout << "\n[4/6] Verification par analyse de contours..." << endl;
```

```

85
86     vector<bool> verificationResults;
87     int confirmedCount = 0;
88
89     for (size_t i = 0; i < boxes.size(); i++) {
90         // Extraction de la region d'interet
91         Rect box = boxes[i];
92         Mat roi = image(box);
93
94         // Analyse de la structure verticale
95         bool isVertical = hasStrongVerticalStructure(roi);
96         verificationResults.push_back(isVertical);
97
98         if (isVertical) {
99             confirmedCount++;
100         }
101
102         // Affichage de progression
103         cout << "         Detection " << (i+1) << "/" << boxes.size()
104             << ": " << (isVertical ? "CONFIRMEE" : "DOUTEUSE") << endl;
105     }
106
107     // =====
108     // PHASE 6: VISUALISATION DES RESULTATS
109     // =====
110
111     cout << "\n[5/6] Generation de l'image de resultat..." << endl;
112
113     Mat result = image.clone();
114
115     for (size_t i = 0; i < boxes.size(); i++) {
116         Rect box = boxes[i];
117         float conf = confidences[i];
118         bool verified = verificationResults[i];
119
120         // Choix de la couleur: vert si confirme, orange sinon
121         Scalar color = verified ? Scalar(0, 255, 0) : Scalar(0, 165,
255);
122
123         // Dessin du rectangle
124         rectangle(result, box, color, 2);
125
126         // Creation de l'etiquette
127         string label = format("Pieton %d: %.1f%%", i+1, conf * 100);
128         if (!verified) {
129             label += " (a verifier)";
130         }
131
132         // Calcul de la position du texte
133         int baseline;
134         Size textSize = getTextSize(label, FONT_HERSHEY_SIMPLEX, 0.5, 1,
&baseline);
135
136         // Dessin du fond de texte
137         Point textOrg(box.x, box.y - 5);
138         rectangle(result,
139             textOrg + Point(0, baseline),
140             textOrg + Point(textSize.width, -textSize.height),

```

```

141         color, FILLED);
142
143         // Dessin du texte
144         putText(result, label, textOrg,
145                 FONT_HERSHEY_SIMPLEX, 0.5, Scalar(255,255,255), 1);
146     }
147
148     // =====
149     // PHASE 7: SAUVEGARDE ET AFFICHAGE
150     // =====
151
152     cout << "\n[6/6] Sauvegarde du resultat..." << endl;
153
154     // Creation du dossier results si necessaire
155     system("mkdir -p results");
156
157     // Sauvegarde de l'image
158     imwrite(outputPath, result);
159     cout << "        Image sauvegardee: " << outputPath << endl;
160
161     // Affichage du resume
162     cout << "\n
                                     n
" << endl;
163     cout << "                RESUME FINAL                " << endl;
164     ;
165     cout << "\n" << endl;
166     cout << "    Total detections YOLO : " << boxes.size() << endl;
167     cout << "    Confirmations          : " << confirmedCount << endl;
168     cout << "    Douteuses              : " << (boxes.size() -
confirmedCount) << endl;
169     cout << "    Taux de confirmation   : "
<< (boxes.empty() ? 0 : (confirmedCount * 100 / boxes.size()))
<< "%\n" << endl;
170
171     // Affichage interactif
172     cout << "Appuyez sur une touche pour afficher l'image..." << endl;
173     waitKey(0);
174
175     imshow("Detection de Pietons - Vert: Confirme, Orange: Douteux",
result);
176     waitKey(0);
177     destroyAllWindows();
178
179     cout << "\nTraitement termine avec succes !" << endl;
180
181     return 0;
182 }

```

Listing 5 – main() - Fonction principale

5 Compilation et Exécution

5.1 Compilation avec CMake

Créez un fichier CMakeLists.txt :

```
1 cmake_minimum_required(VERSION 3.10)
2 project(DetectionPietons)
3
4 set(CMAKE_CXX_STANDARD 11)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # Recherche d'OpenCV
8 find_package(OpenCV REQUIRED)
9 include_directories(${OpenCV_INCLUDE_DIRS})
10
11 # Ajout de l'executable
12 add_executable(detecteur_pietons src/detection.cpp)
13
14 # Liaison avec les bibliotheques
15 target_link_libraries(detecteur_pietons ${OpenCV_LIBS})
16
17 # Options de compilation
18 set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -O3 -march=native")
```

Listing 6 – CMakeLists.txt

Commandes de compilation :

```
1 mkdir build
2 cd build
3 cmake ..
4 make -j4
```

5.2 Compilation Directe

```
1 g++ -std=c++11 src/detection.cpp -o detecteur_pietons \
2     'pkg-config --cflags --libs opencv4' \
3     -O3 -march=native
```

5.3 Exécution

```
1 # Execution simple
2 ./detecteur_pietons data/test.jpg
3
4 # Avec fichier de sortie specifique
5 ./detecteur_pietons data/test.jpg results/mon_resultat.png
6
7 # Traitement de plusieurs images (script bash)
8 for img in data/*.jpg; do
9     ./detecteur_pietons $img results/${basename $img}
10 done
```

6 Exercices et Analyses

6.1 Exercice 1 : Étude des Paramètres de Sobel

1. Modifiez le seuil de binarisation (Config : :SOBEL_THRESHOLD) :
 - Valeurs à tester : 30, 50, 70, 100
 - Observez l'impact sur les détections confirmées
 - Calculez le nombre de contours détectés en fonction du seuil
2. Changez la pondération dans addWeighted :
 - Testez (0.8, 0.2) : favorise les contours verticaux
 - Testez (0.2, 0.8) : favorise les contours horizontaux
 - Quelle configuration donne les meilleurs résultats pour les piétons ?

6.2 Exercice 2 : Optimisation des Paramètres Hough

```
1 HoughLinesP(edges, lines, rho, theta, threshold, minLineLength,
    maxLineGap);
```

Listing 7 – Paramètres de HoughLinesP

Étudiez l'impact de chaque paramètre :

Paramètre	Rôle	Valeurs à tester	Effet observé
threshold	Nb minimum de votes	30, 50, 70, 100	
minLineLength	Longueur min des segments	20, 35, 50, 80	
maxLineGap	Écart max entre segments	5, 12, 20, 30	

TABLE 1 – Analyse des paramètres de Hough

6.3 Exercice 3 : Métriques de Performance

Implémentez une fonction qui calcule :

1. **Précision** : $P = \frac{VP}{VP+FP}$
2. **Rappel** : $R = \frac{VP}{VP+FN}$
3. **F1-score** : $F1 = 2 \cdot \frac{P \cdot R}{P+R}$

où :

- VP = Vrais Positifs (piétons correctement détectés)
- FP = Faux Positifs (non-piétons détectés comme piétons)
- FN = Faux Négatifs (piétons non détectés)

6.4 Exercice 4 : Détection de Têtes par HoughCircles

Ajoutez une vérification complémentaire pour les têtes :

```

1 bool hasHeadStructure(const Mat& roi) {
2     // Extraire la partie superieure de la ROI (30% superieurs)
3     Rect headRegion(0, 0, roi.cols, roi.rows / 3);
4     Mat head = roi(headRegion);
5
6     // Convertir en niveaux de gris
7     Mat gray;
8     cvtColor(head, gray, COLOR_BGR2GRAY);
9
10    // Detection de cercles avec HoughCircles
11    vector<Vec3f> circles;
12    HoughCircles(gray, circles, HOUGH_GRADIENT, 1, 20,
13                100, 30, roi.cols/6, roi.cols/3);
14
15    return !circles.empty();
16 }

```

Listing 8 – Détection de cercles avec HoughCircles

7 Annexes Techniques

7.1 Annexe A : Classes COCO

ID	Classe	ID	Classe
0	personne	40	bouteille
1	bicyclette	41	verre
2	voiture	42	fourchette
3	moto	43	couteau
4	avion	44	cuillère
5	bus	45	bol
6	train	46	banane
7	camion	47	pomme
8	bateau	48	sandwich
9	feu de circulation	49	orange
10	bouche d'incendie	50	brocoli

TABLE 2 – Principales classes du jeu de données COCO

7.2 Annexe B : Codes d'Erreur OpenCV

Code	Signification
-215	Assertion failed (fichier non trouvé)
-216	Format de fichier non supporté
-217	Erreur de lecture/écriture
-218	Erreur de mémoire insuffisante
-219	Erreur OpenCL

TABLE 3 – Codes d'erreur courants

7.3 Annexe C : Optimisations Possibles

1. Accélération GPU :

```
1 net.setPreferableBackend(DNN_BACKEND_CUDA);  
2 net.setPreferableTarget(DNN_TARGET_CUDA);  
3
```

2. Traitement parallèle : Utilisation d'OpenMP

```
1 #pragma omp parallel for  
2 for (int i = 0; i < boxes.size(); i++) {  
3     // Traitement parallele des ROIs  
4 }  
5
```

3. Réduction de la résolution : Traitement pyramidal

8 Conclusion

Perspectives :

- Extension à la vidéo (détection temps réel)
- Intégration d'un tracker (DeepSORT, Kalman)
- Fine-tuning du modèle YOLO sur des données spécifiques
- Déploiement sur systèmes embarqués (Jetson Nano, Raspberry Pi)